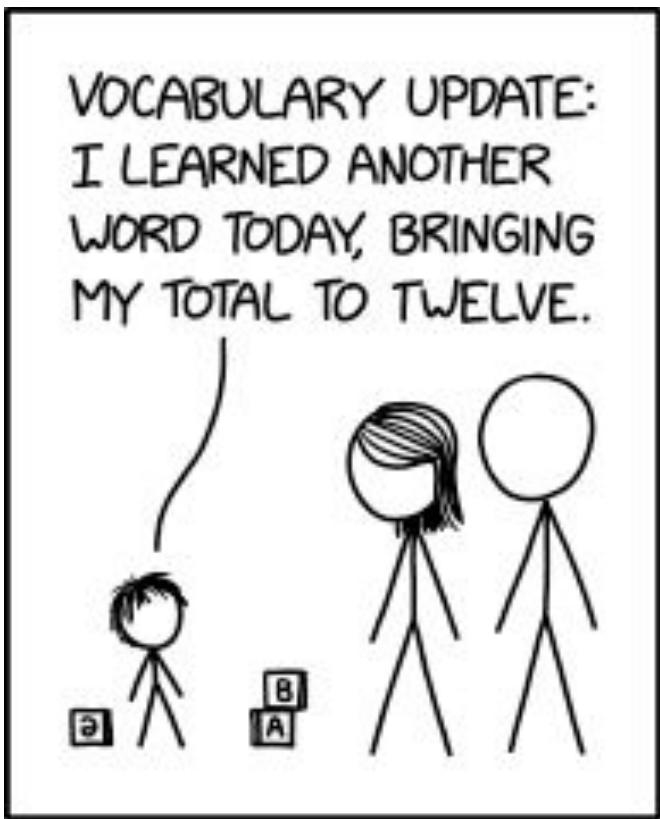


CS 159 Week 2:

Tokenization, Segmentation, and Evaluation



(<https://xkcd.com/2839>)

Announcements

- Canvas has been set up...I think? (reminder: ONLY for announcements and discussions)
- Everyone should also have access to Gradescope (let me know if not)
- Look for an email from Nic Dodds in the next two days with info about logging in to the course server
- Office and tutoring hours will be announced on or before Thursday (and will start then, with the release of Lab 1)

Reminder: Theme of Weeks 1-2: this clip, but replace “birds” with “words”

Where we left off:



A word is a string! Like “strawberry”.

Can’t a string have multiple words? Like, the whole text of Moby Dick. But also: what’s a string?



A string is an array of characters. So “strawberry” is actually [‘s’, ‘t’, ‘r’, ‘a’, ‘w’, ‘b’, ‘e’, ‘r’, ‘r’, ‘y’].

Cool, cool. ...And a character is what, exactly?



Where we left off:



A word is a string! Like “strawberry”.

Can't a string have multiple words? Like, the whole text of Moby Dick. But also: what's a string?



A string is an array of characters. So “strawberry” is actually ['s', 't', 'r', 'a', 'w', 'b', 'e', 'r', 'r', 'y'].



Cool, cool. ...And a character is what, exactly?



And now, the continuation...



A word is a string! Like “strawberry”.

Can't a string have multiple words? Like, the whole text of Moby Dick. ~~But also: what's a string?~~



A string is an array of characters. So “strawberry” is actually ['s', 't', 'r', 'a', 'w', 'b', 'e', 'r', 'r', 'y'].

Cool, cool. ...And a character is what, exactly?



Is this a word?

“strawberry”

Is this a word?

“strawberry jam”

Is this a word?

“strawberry and”

Is this a word?

“berry”

Is this a word?

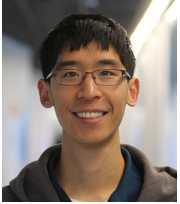
“rry”

Is this a word?

“r”

This is getting too complicated! I just want an adult to tell me “‘strawberry’ is a word because I said so.”





This is getting too complicated! I just want an adult to tell me “‘strawberry’ is a word because I said so.”



Sounds reasonable to me!

Remember one of the core themes of CS 159: *Any system is made up of decisions made by human beings.*

One such decision is what things get counted as words!

Some terminology:

Vocabulary: a set V containing everything that we consider to be a word (for a given application)

- Example: $V = \{\text{“strawberry”, “jam”, “berry”}\}$

Type: any element $y \in V$

Why “types”?

Consider vocabulary $V = \{\text{“strawberry”, “jam”, “berry”}\}$

How many *words* are there in the following text?

berry berry berry berry

Why “types”?

Consider vocabulary $V = \{\text{“strawberry”, “jam”, “berry”}\}$

How many *types* are there in the following text?

berry berry berry berry

Reminder: a *type* is defined as a member of a set, and is therefore unique (no repeats)

Some terminology (continued):

Vocabulary: a set V containing everything that we consider to be a word (for a given application)

- Example: $V = \{\text{“strawberry”, “jam”, “berry”}\}$

Type: any element $y \in V$

Some terminology (continued):

Vocabulary: a set V containing everything that we consider to be a word (for a given application)

- Example: $V = \{\text{"strawberry"}, \text{"jam"}, \text{"berry"}\}$

Type: any element $y \in V$

Instance: any literal occurrence of a type y in a given text

- Example: "berry berry berry berry" has 4 instances of the type "berry".

Neat! We've defined our way out of the question "what are words?"

Neat! We've defined our way out of the question "what are words?"

Problem: We're typically not explicitly given a bunch of word instances; we're given raw text that may contain word instances.

Neat! We've defined our way out of the question "what are words?"

Problem: We're typically not explicitly given a bunch of word instances; we're given raw text that may contain word instances.

You know, like I said earlier: a string containing the entire text of Moby Dick...



I teach at a university

I teach at a university

Meh, seems straightforward enough, just break it up by whitespace!



I teach at a university

Meh, seems straightforward enough, just break it up
by whitespace!



我在大学教书

I teach at a university

Meh, seems straightforward enough, just break it up
by whitespace!



我在大学教书

Noooooooo, what have you done to my spaces????
...Fine, maybe we can work at the character level?



我在大学教书

我在大学教书

Big, great

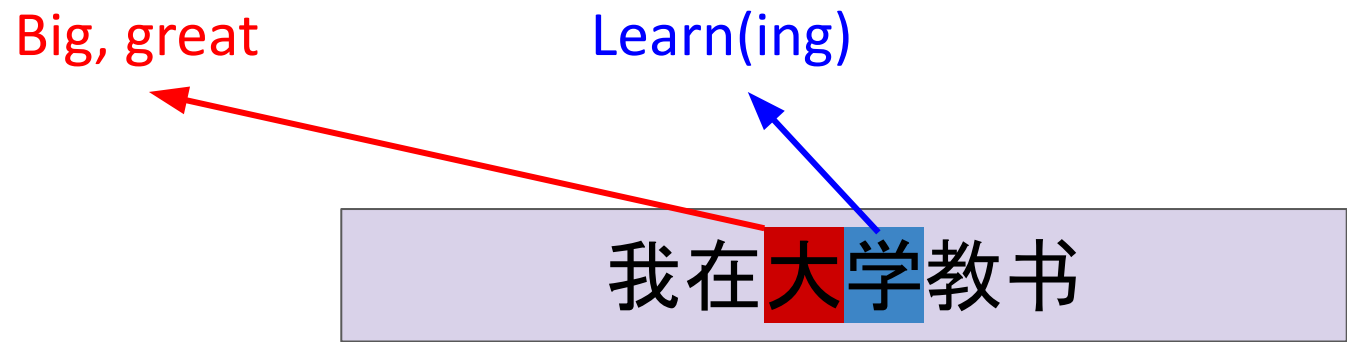


我在大学教书

Big, great

Learn(ing)

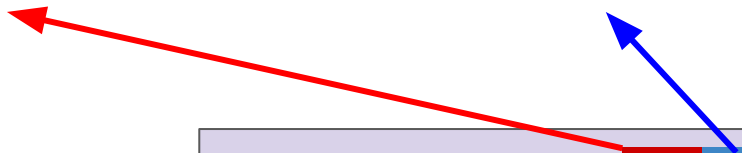
我在大学教书



The diagram illustrates semantic associations from a Chinese sentence to English phrases. A light purple rectangular box contains the Chinese text '我在大学教书' (I teach at university). The character '大' (big) is highlighted with a red background, and the character '学' (study/learn) is highlighted with a blue background. A red arrow originates from the red '大' and points to the English text 'Big, great' located above and to the left of the box. A blue arrow originates from the blue '学' and points to the English text 'Learn(ing)' located above and to the right of the box.

Big, great + Learn(ing) = Big Learning?

我在大学教书



我在大学教书

university

Extracting meaning from raw text usually requires breaking the text down into more “fundamental units”

But what exactly are the “fundamental units” of text processing? As we’ve seen, we have a few options...

Options for the “fundamental unit” of text processing

Individual characters

e.g., “r”

Words (as defined by some vocabulary)

e.g., “strawberry”

Options for the “fundamental unit” of text processing

Individual characters

e.g., “r”

Sequences of characters (for some languages, these are known as *subwords*)

e.g., “大学”

Words (as defined by some vocabulary)

e.g., “strawberry”

Options for the “fundamental unit” of text processing

Individual characters

e.g., “r”

Sequences of characters (for some languages, these are known as *subwords*)

e.g., “大学”

Words (as defined by some vocabulary)

e.g., “strawberry”

Sequences of words (aka *n-grams*)

e.g., (“strawberry”, “jam”),
representing “strawberry jam”

Options for the “fundamental unit” of text processing

Individual characters

e.g., “r”

Sequences of characters (for some languages, these are known as *subwords*)

e.g., “大学”

Words (as defined by some vocabulary)

e.g., “strawberry”

Sequences of words (aka *n-grams*)

e.g., (“strawberry”, “jam”),
representing “strawberry jam”

A fundamental unit (whichever option you pick) is called a **token**, and the process of breaking down text into tokens is called **tokenization**.

Two approaches to tokenization

Subword tokenization

e.g., “r”

e.g., “大学”

Rule-based tokenization

e.g., “strawberry”

e.g., (“strawberry”, “jam”),
representing “strawberry jam”

Two approaches to tokenization

Subword tokenization

(good for these!)

e.g., “r”

e.g., “大学”

Rule-based tokenization

(good for these!)

e.g., “strawberry”

e.g., (“strawberry”, “jam”),
representing “strawberry jam”

Two approaches to tokenization

Subword tokenization

(this week's reading)

(good for these!)

e.g., "r"

e.g., "大学"

Rule-based tokenization

(right now)

(good for these!)

e.g., "strawberry"

e.g., ("strawberry", "jam"),
representing "strawberry jam"

Rule-based tokenization

Classical approach to tokenization (around since at least the 20th century); explored both in CS and linguistics.

Splits text into tokens by following hand-written rules, usually designed based on human insights about the language.

- E.g., in English, we know that words *tend* to be separated by whitespace.

In CS, we generally implement these rules in code.

Implementing tokenization rules in code

Tokenization rules can get quite complex...but last week, we saw a great tool for *expressing* complicated text operations in a concise way!

Implementing tokenization rules in code

Tokenization rules can get quite complex...but last week, we saw a great tool for *expressing* complicated text operations in a concise way!

Regular expressions!

“In English, words tend to be separated by whitespace”

“In English, words tend to be separated by whitespace”

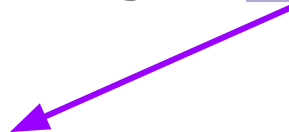


RETURN: all matches for the regex `\S+`

“In English, words tend to be separated by whitespace”



RETURN: all matches for the regex `\S+`



Non-whitespace character



One or more

Is regex enough?

Input text

“I teach at a university”

Matches from \S+

[“I”, “teach”, “at”, “a”,
“university”]

Is regex enough?

Input text

“I teach at a university”

“O.K., it’s (maybe) not so simple...”

Matches from \S+

[“I”, “teach”, “at”, “a”,
“university”]

[“O.K.”, “it’s”, “(maybe)”,
“not”, “so”, “simple...”]

Is regex enough?

Input text

“I teach at a university”

“O.K., it’s (maybe) not so simple...”

Matches from \S+

[“I”, “teach”, “at”, “a”,
“university”]

[“O.K.”, “it’s”, “(maybe)”,
“not”, “so”, “simple...”]

In practice, regex can be used as a good *first pass*, but for many use cases we may need to follow it up with some manual if-then logic.

Is regex enough?

Input text

“I teach at a university”

“O.K., it’s (maybe) not so simple...”

Matches from \S+

[“I”, “teach”, “at”, “a”,
“university”]

[“O.K.”, “it’s”, “(maybe)”,
“not”, “so”, “simple...”]

In practice, regex can be used as a good *first pass*, but for many use cases we may need to follow it up with some manual if-then logic.

Example: “if a match begins or ends with a non-alphanumeric symbol, strip it out.”

Activity: Your turn!

- Assume we have started by collecting all matches for the regex `\S+`
- In groups of 3-4: come up with additional follow-up rules to account for (what you think are) common edge cases missed by that first pass.
- You don't need to express the rules in code, you can just describe them in plain English.
- Feel free to use a regex tester to see what `\S+` does and doesn't match.
- When finished: one person from each group can submit what you came up with at the following form:

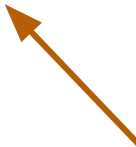


But wait, there's more!

“I teach at a university.”

But wait, there's more!

"I teach at a university."



Is this the end of a sentence?

But wait, there's more!

“I teach at a university. You need a Ph.D. to do that.”

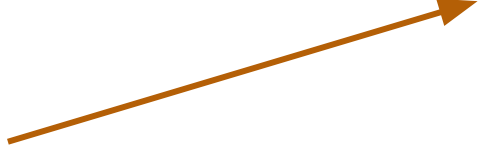


What about these?

But wait, there's more!

“I teach at a university. That was my motivation for getting a Ph.D.”

How about now?



Some other “weird” cases...

- Jim says “Hello.” Also Kate does too.
- “Glad to be of help,” the agent said. “Now do you have any other questions?”
- What?!???
- Imagine this is a sentence

And this is the next one in the same document

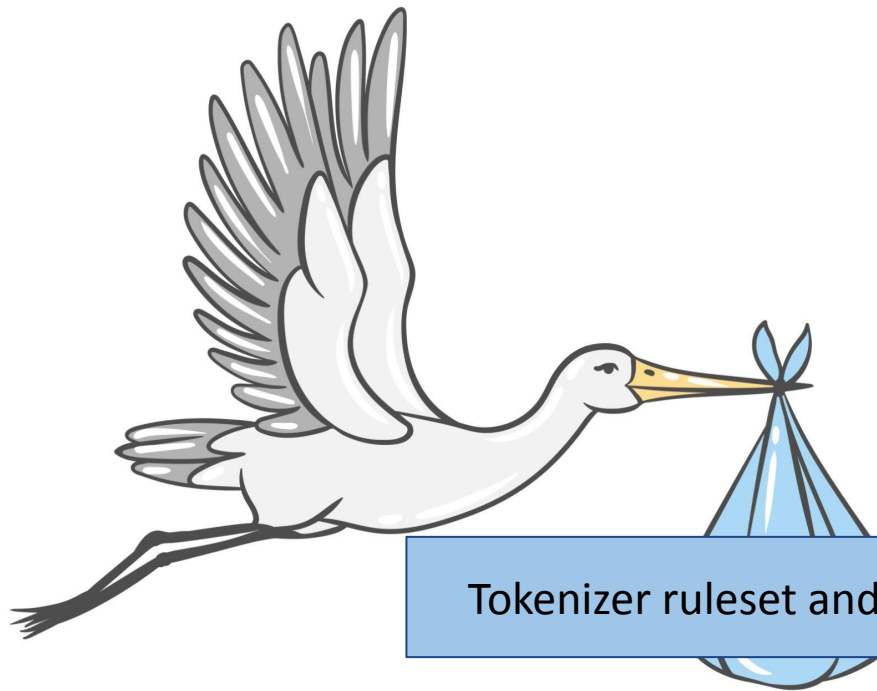
From *tokenization* to *segmentation*

The process of splitting a piece of text (or a token sequence) into sentences is called **segmentation**.

Like tokenization, (rule-based) segmentation often works by starting simple (e.g., always splitting on punctuation marks) then adding extra logic to account for special cases.

You'll get a chance to practice this in Lab 1!

Where do rules, and vocabularies, come from?



What “should” count as a word is highly context dependent

Many words are domain-specific:

- Technical terms, e.g., “neutrino”
- Slang, e.g., “brainrot”
- Dialects, e.g., “y’all”
- Fictional terms, e.g., “Ent”
- Code-switching in multilingual communities, e.g., “哎呀! This is bad!”

Corpora

Every NLP system has some intended use case, or target audience.

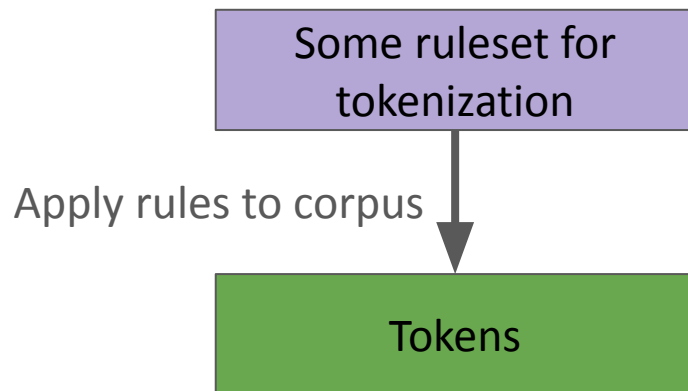
We generally develop our vocabulary and tokenizer rules by looking at some *reference material* that is representative of our use case.

This reference material usually takes the form of a curated collection of text known as a **corpus**.

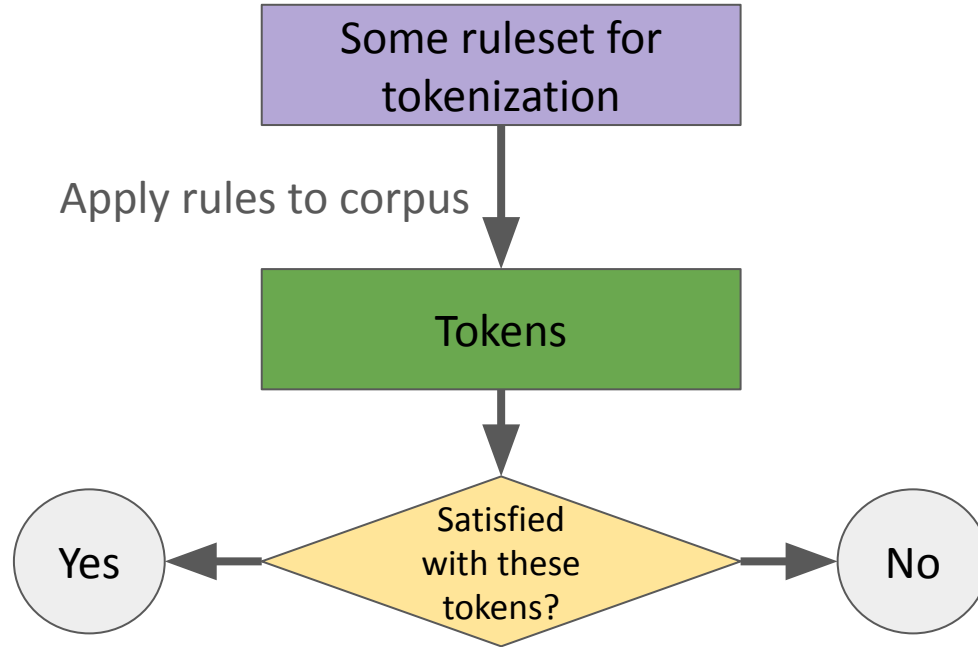
Developing a vocabulary and rules is an iterative process

Some ruleset for
tokenization

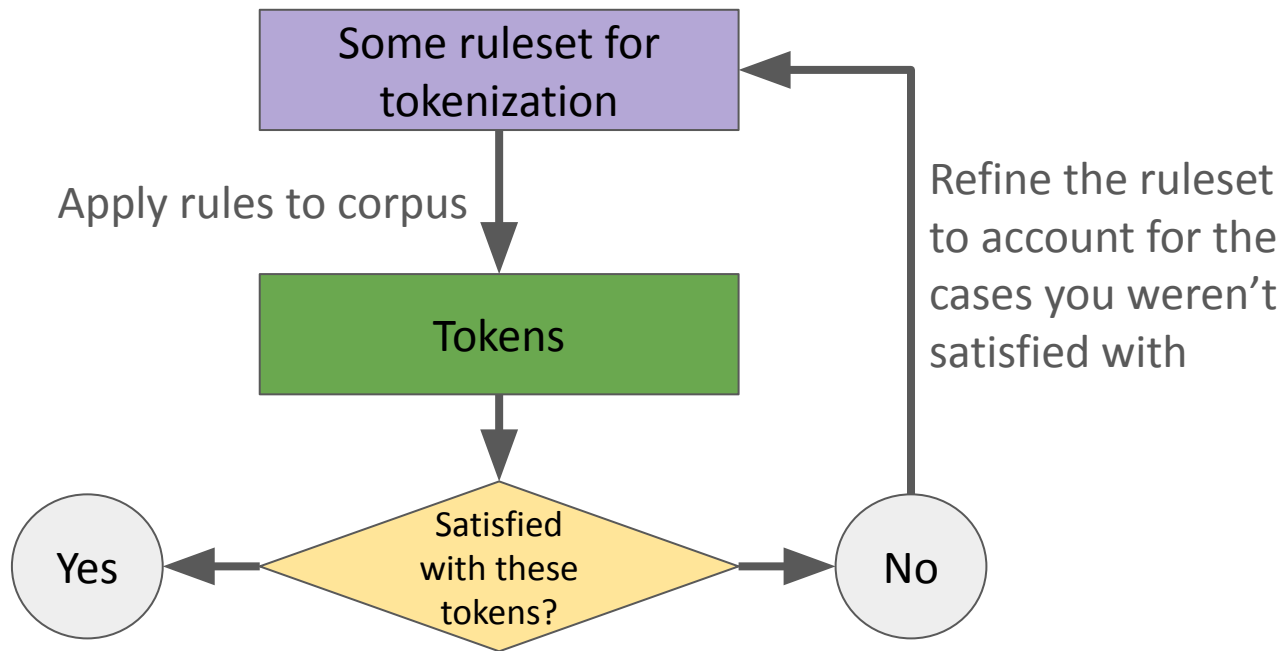
Developing a vocabulary and rules is an iterative process



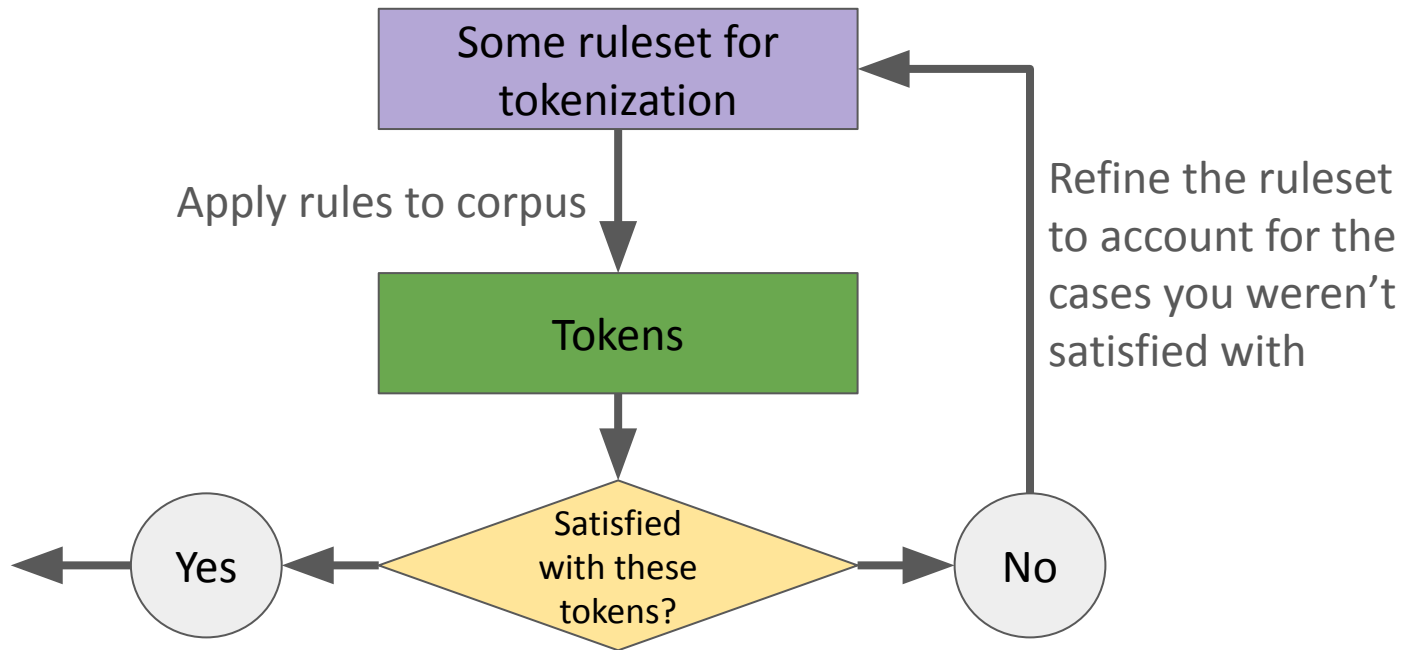
Developing a vocabulary and rules is an iterative process



Developing a vocabulary and rules is an iterative process



Developing a vocabulary and rules is an iterative process



Done (for now!)
Keep this **ruleset**,
and the set of
resulting **tokens** as
your vocabulary.

Places to find corpora

For the lab assignments, corpora will be provided for you...

...But for your final project, you may be wondering how to find a good corpus to use.

Thankfully, people have done the hard work of constructing text corpora for a variety of use cases, and collecting them in repositories:

- [UCI Machine Learning Repository](#)
- [HuggingFace Datasets Index](#)
- Many NLP libraries package their own datasets (shout-out to [ConvoKit](#) from my old lab!)

Considerations for responsible corpus usage:

- Is this corpus actually representative of my intended use case? My intended audience?
- Is my use of this corpus consistent with the creators'/authors' intent?
- Any *legal* limitations regarding how this corpus can be used? (May vary by country!)
- Did the creators of this corpus break any laws (or act unethically) in the creation of the corpus? (See: recent lawsuits over Anthropic, Meta, etc.'s use of pirated material in LLM training)
- Look for a **datasheet** accompanying the corpus, it should help answer most of these questions.
- Similar concerns hold when collecting your own data; in this case, consider questions of consent and terms of service, and (for research projects) consult the IRB.

End-of-class reminders:

Post-lecture readings are listed on the website (this week: J&M 2.4 (up until 2.4.3); 4.9)

Lab 1 goes out on Thursday; class will be a work session

Before coming to Thursday's class, fill out the Week 2 check-in on Gradescope